CSA0613 - Design and Analysis of Algorithms

CO4-AT3-Comparative Analysis

Divyesh S P (192424147).

Knapsack Problem Comparison

0/1 Dynamic Programming vs Fractional Greedy

1. Aim

To solve the Knapsack Problem using **0/1 Dynamic Programming** and **Fractional Greedy Algorithm**, and compare their values, selection strategies, and computational complexities.

2. Problem Statement

Given:

- **Weights:** [10, 20, 30]
- **Values:** [60, 100, 120]
- **Capacity:** 50

Find the maximum possible value that can be carried.

3. Input

Weights = [10, 20, 30]

Values = [60, 100, 120]

Capacity = 50

4. 0/1 Knapsack – Dynamic Programming

In **0/1 Knapsack**, an item can either be:

- Completely selected, or
- Completely rejected.

An item **cannot be divided**.

Python Code

```
weights = [10, 20, 30]
values = [60, 100, 120]
capacity = 50

n = len(weights)

dp = [[0] * (capacity + 1) for _ in range(n + 1)]

for i in range(1, n + 1):
    for w in range(capacity + 1):

        if weights[i - 1] <= w:
            dp[i][w] = max(
                values[i - 1] + dp[i - 1][w - weights[i - 1]],
                dp[i - 1][w]
            )
        else:
            dp[i][w] = dp[i - 1][w]

# Find selected items
w = capacity
selected = []

for i in range(n, 0, -1):
```

```
    if dp[i][w] != dp[i - 1][w]:
        selected.append(i)
        w -= weights[i - 1]

selected.reverse()

print("===== 0/1 KNAPSACK =====")
print("Maximum Value:", dp[n][capacity])
print("Selected Items:", selected)
```

Output

```
===== 0/1 KNAPSACK =====
Maximum Value: 220
Selected Items: [2, 3]
```

5. Fractional Knapsack – Greedy

In **Fractional Knapsack**, an item can be divided into smaller portions.

The algorithm calculates:

Value/WeightValue/Weight

and selects items with the highest ratio first.

Python Code

```
weights = [10, 20, 30]
values = [60, 100, 120]
capacity = 50

items = []

for i in range(len(weights)):
```

```
    ratio = values[i] / weights[i]
    items.append((ratio, weights[i], values[i], i + 1))

items.sort(reverse=True)

total_value = 0
remaining = capacity
selected = []

for ratio, weight, value, item in items:

    if remaining >= weight:
        remaining -= weight
        total_value += value
        selected.append((item, weight))
    else:
        fraction = remaining / weight
        total_value += value * fraction
        selected.append((item, remaining))
        break

print("===== FRACTIONAL KNAPSACK =====")
print("Maximum Value:", total_value)
print("Selected Items:", selected)
```

Output

```
===== FRACTIONAL KNAPSACK =====
Maximum Value: 240.0
Selected Items: [(1, 10), (2, 20), (3, 20)]
```

6. Comparison

Feature	**0/1 Knapsack**	**Fractional Knapsack**
Technique	Dynamic Programming	Greedy
Item division	Not allowed	Allowed
Selected items	Items 2 and 3	Items 1, 2 and part of 3
Maximum Value	**220**	**240**
Time Complexity	O(n × W)	O(n log n)
Space Complexity	O(n × W)	O(n)
Strategy	Considers combinations	Highest value/weight first

7. Why Greedy Works for Fractional Knapsack

For fractional knapsack, items can be divided.

The value-to-weight ratios are:

Item 1: 60 / 10 = 6

Item 2: 100 / 20 = 5

Item 3: 120 / 30 = 4

Therefore, Greedy selects:

Item 1→ 10 kg

Item 2 → 20 kg

Item 3 → 20 kg

The remaining 20 kg of capacity can be filled with part of Item 3.

This gives the optimal value of:

60+100+80=24060 + 100 + 80 = \boxed{240}

8. Why Greedy Does Not Always Work for 0/1 Knapsack

In 0/1 Knapsack, items **cannot be divided**.

A locally best value/weight choice may prevent a better combination of complete items.

Therefore, Greedy does not guarantee an optimal solution for the general 0/1 Knapsack problem.

Dynamic Programming considers different combinations and guarantees the optimal result.

For this example:

0/1 → Item 2 + Item 3
→ Weight = 50
→ Value = 220

Fractional → Item 1 + Item 2 + 2/3 of Item 3
→ Weight = 50
→ Value = 240

9. Complexity Analysis

0/1 Knapsack

Time complexity:

$O(nW)$

where:

- n = number of items
- W = knapsack capacity

Space complexity:

$O(nW)$

The DP table requires additional memory.

Fractional Knapsack

Sorting the items according to value/weight ratio takes:

$O(n \log n)$

The selection takes:

$O(n)$

Therefore:

Space complexity is approximately:

$O(n)$

10. Memory Usage in DP

The 0/1 Knapsack algorithm creates a DP table of size:

$(n + 1) \times (capacity + 1)$

For this problem:

$4 \times 51 = 204$ entries

As the capacity increases, the memory requirement also increases.

This is one of the main limitations of the DP approach.

11. Resource Allocation Insight

The appropriate approach depends on the type of resource allocation.

Use 0/1 Dynamic Programming when:

- Resources/items cannot be divided.
- Each item must be fully selected or rejected.
- An exact optimal solution is required.

Use Fractional Greedy when:

- Resources can be divided.
- Partial allocation is allowed.
- A fast solution is required.

Examples:

0/1: Selecting complete projects, machines, or equipment.

Fractional: Allocating fuel, raw materials, storage space, or divisible resources.

12. Result

Result: The Knapsack Problem was successfully implemented using both **0/1 Dynamic Programming** and **Fractional Greedy Algorithm**. The 0/1 approach obtained a maximum value of **220**, while the Fractional approach obtained **240**. The experiment shows that Greedy provides an optimal solution when items can be divided, whereas Dynamic Programming is required to guarantee an optimal solution when items must be selected completely.